



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Modern Compiler Design

Subject Code – BE103701

Faculty Name – Radhika Gokani

CHAPTER -3 Syntax Analysis

1. Understanding Parser and CFG(Context Free Grammars)

A **Context-Free Grammar** is a formal system used to define the syntax of programming languages and other structured languages.

Components of a CFG:

A CFG is defined as a 4-tuple: $G = (V, T, P, S)$ Where:

- **V**: Set of non-terminal symbols (e.g., S, A, B)
- **T**: Set of terminal symbols (e.g., a, b, +, *)
- **P**: Set of production rules (e.g., $S \rightarrow aSb \mid \epsilon$)
- **S**: Start symbol (usually the main non-terminal like S)

Example:

Code

$S \rightarrow aSb \mid \epsilon$

This grammar generates strings like ab, aabb, aaabbb, etc.

Features:

- Defines **structure**, not meaning.
- Used in **compiler design**, **natural language processing**, and **syntax analysis**.
- Can be **ambiguous**, meaning multiple parse trees for the same string.

What Is a Parser?

A **Parser** is a tool or algorithm that takes input (usually a string of tokens) and determines if it can be generated by a given grammar—often producing a **parse tree**.

Types of Parsers:

- **Top-Down Parsers**: Start from the start symbol and try to rewrite it to match the input.
 - Example: Recursive Descent Parser
- **Bottom-Up Parsers**: Start from the input and try to reduce it to the start symbol.
 - Example: Shift-Reduce Parser

Parse Tree:

A hierarchical tree that shows how a string is derived from the grammar. Each node represents a rule application.

Use Cases:

- **Compilers**: Convert source code into syntax trees.
- **Interpreters**: Analyze and execute code.
- **Natural Language Processing**: Understand sentence structure.

2. Left Recursion and Left Factoring of grammar

Left Recursion

Left recursion occurs when a non-terminal in a grammar refers to itself as the first symbol on the right-hand side of a production rule.

Example:

Code

$A \rightarrow A\alpha \mid \beta$

Here, A is left-recursive because it appears first on the right-hand side.

Why It's a Problem:

- **Top-down parsers** (like recursive descent parsers) can't handle left recursion—they go into infinite loops.

How to Eliminate Left Recursion:

Transform the grammar to remove direct left recursion:

Original:

Code

$A \rightarrow A\alpha \mid \beta$

Transformed:

Code

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

This introduces a new non-terminal A' and uses right recursion instead.

Left Factoring

Left factoring is a grammar transformation used when two or more productions for a non-terminal begin with the same prefix. It helps parsers decide which rule to apply by factoring out the common part.

Example:

Code

$A \rightarrow \alpha\beta 1 \mid \alpha\beta 2$

Left-Factored Version:

Code

$A \rightarrow \alpha A'$

$A' \rightarrow \beta 1 \mid \beta 2$

This makes the grammar more suitable for **predictive parsing**, where decisions are made based on lookahead tokens.

Table

Concept	Problem It Solves	Technique Used
Left Recursion	Infinite loops in top-down parsers	Replace with right recursion
Left Factoring	Ambiguity in choosing production rules	Factor out common prefixes

3. Top Down and Bottom up Parsing Algorithms

Top-Down Parsing

Top-down parsing starts from the **start symbol** of a grammar and tries to derive the input string by applying production rules. It builds the **parse tree from the root to the leaves**, using **leftmost derivation**.

Top-Down Parsing Algorithm (Recursive Descent)

Step-by-Step:

1. **Start with the start symbol** of the grammar.
2. **For each non-terminal**, write a function that:
 - Tries each production rule.
 - Recursively calls functions for other non-terminals.
 - Matches terminals with input tokens.
3. **Advance input pointer** when terminals match.
4. **Backtrack** if no rule matches (unless using lookahead).
5. **Accept** if the entire input is consumed and matches the grammar.

Example Grammar

Code

```
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → ( E ) | id
Input: id + id * id
```

The parser would:

- Start with E
- Expand to T E'
- Match id in F
- Continue through T', E', and so on...

Advantages

- Simple and intuitive.
- Easy to implement manually.
- Good for small grammars.

Limitations

- **Cannot handle left recursion.**
- May require **left factoring**.
- Not suitable for all grammars.

4. Operator-Precedence Parsing

Operator-Precedence Parsing is a type of **bottom-up parsing** technique used to handle expressions involving operators (like +, *, /, etc.) based on their **precedence** and **associativity**. It's particularly useful for arithmetic expressions and is simpler than full LR parsing.

Concepts

1. Operator Precedence

Each operator has a **priority** (precedence) and **direction** (associativity):

- * and / have higher precedence than + and -.
- Associativity defines how operators of the same precedence are grouped:
 - Left-associative: $a - b - c \rightarrow (a - b) - c$
 - Right-associative: $a \wedge b \wedge c \rightarrow a \wedge (b \wedge c)$

2. Operator-Precedence Table

A table defines relationships between operators:

- < means shift (move input to stack)
- > means reduce (apply production)
- = means equal precedence (often between matching parentheses)

	+	*	(	)	id
+	>	<	<	>	<
*	>	>	<	>	<
(	<	<	<	=	<
)	>	>		>	
id	>	>		>	

Parsing Steps

1. **Initialize** stack with a special symbol \$.
2. **Scan input** from left to right.
3. Use the **precedence table** to decide:
 - **Shift**: push input symbol onto stack.
 - **Reduce**: apply grammar rule to replace symbols on stack.
4. Repeat until input is consumed and stack contains only the start symbol.

Example Grammar

Code

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Input: id + id * id

Using the precedence table, the parser will:

- Shift id
- Reduce to E
- Shift +
- Shift id
- Reduce to E
- Shift *
- Shift id
- Reduce to E
- Reduce E * E
- Reduce E + E
- Accept

Advantages

- Efficient for expressions with operators.
- Simple to implement for arithmetic grammars.
- No need for full parsing tables.

Limitations

- Only works for grammars with **operator precedence relations**.
- Cannot handle general context-free grammars.
- Requires careful construction of precedence table.

5. LR Parsers

LR stands for:

- **L**: Left-to-right scanning of the input.
- **R**: Producing a **rightmost derivation in reverse**.

LR parsers read the input from left to right and construct the parse tree from the leaves up to the root, reducing substrings to non-terminals using grammar rules.

LR Parsing Works

LR parsers use a **stack** and a **parsing table** to decide whether to:

- **Shift**: Push the next input symbol onto the stack.
- **Reduce**: Replace a sequence of symbols on the stack with a non-terminal.
- **Accept**: Successfully parsed the input.
- **Error**: Input doesn't match the grammar.

Types of LR Parsers

Parser Type	Description	Power
LR(0)	No lookahead. Simplest form.	Limited
SLR(1)	Simple LR with 1-token lookahead. Uses FOLLOW sets.	Moderate
LALR(1)	Look-Ahead LR. Combines states with same items.	Widely used
Canonical LR(1)	Full LR with 1-token lookahead. Most powerful.	High

LR Parser Components

1. **Input Buffer**: Holds the input string.
2. **Stack**: Stores symbols and states.
3. **Parsing Table**:
 - **Action Table**: Tells whether to shift, reduce, accept, or error.
 - **Goto Table**: Tells which state to go to after a reduction.
4. **Start State**: Initial parser state.

Example Workflow

Given input: $id + id * id$

1. Shift id
2. Reduce $id \rightarrow F \rightarrow T \rightarrow E$
3. Shift $+$
4. Shift id
5. Reduce $id \rightarrow F \rightarrow T$
6. Shift $*$
7. Shift id
8. Reduce $id \rightarrow F$
9. Reduce $T * F \rightarrow T$
10. Reduce $E + T \rightarrow E$
11. Accept

Advantages

- Handles **all deterministic context-free grammars**.
- Supports **left recursion**.
- Efficient and deterministic.

Disadvantages

- Complex to implement manually.
- Requires **parser generators** for practical use.

6. Using Ambiguous Grammars

A grammar is **ambiguous** if there exists **at least one string** that can be generated by **more than one distinct parse tree** (or derivation).

Example:

Consider the grammar:

Code

$E \rightarrow E + E \mid E * E \mid \text{id}$

The string `id + id * id` can be parsed in two ways:

- As `(id + id) * id`
- As `id + (id * id)`

This ambiguity arises because the grammar doesn't specify operator precedence or associativity.

ambiguous grammars can be useful in certain contexts:

1. Natural Language Processing (NLP)

- Human languages are inherently ambiguous.
- Ambiguous grammars allow modeling multiple interpretations.
- Parsers can rank or resolve ambiguity using context or probability.

2. High-Level Language Design

- Some languages intentionally allow ambiguous constructs for flexibility.
- Ambiguity is resolved later using semantic rules or precedence tables.

3. Simplified Grammar Definitions

- Ambiguous grammars can be shorter and easier to write.
- Useful for prototyping or educational purposes.

How to Handle Ambiguity

- **Disambiguation Rules:** Define precedence and associativity explicitly.
- **Grammar Refactoring:** Rewrite the grammar to eliminate ambiguity.
- **Parser Design:** Use parser generators that support ambiguity (e.g., GLR parsers).

Summary

Aspect	Ambiguous Grammar
Definition	Multiple parse trees for one string
Problem	Unclear syntax, compiler confusion
Use Cases	NLP, flexible language design
Handling Techniques	Refactoring, precedence rules, GLR

7. Parser Generators

A **parser generator** takes a grammar (usually written in a format like BNF or EBNF) and produces source code for a parser in a target programming language (e.g., C, Java, Python). This parser can then be used to analyze and interpret structured input—like programming languages, configuration files, or domain-specific languages.

How It Works

1. **Define Grammar:** Write grammar rules in a supported format.
2. **Generate Parser:** Run the parser generator to produce source code.
3. **Integrate:** Use the generated parser in your application to analyze input.
4. **Parse Input:** Feed strings to the parser and get parse trees or abstract syntax trees (ASTs).

Parser Generators

Tool	Language Support	Parser Type	Notes
YACC	C	LALR(1)	Classic tool; often paired with Lex for tokenizing.
Bison	C/C++	LALR(1)	GNU version of YACC with more features.
ANTLR	Java, Python, C#, etc.	LL(*)	Very powerful; supports complex grammars and tree walking.
PEG.js	JavaScript	PEG	For building parsers in web apps.
PLY	Python	LALR(1)	Python implementation of Lex and YACC.
Nearley	JavaScript	Earley	Handles ambiguous grammars well.

Advantages

- Saves time and reduces errors.
- Handles complex grammars efficiently.
- Often includes error recovery and debugging tools.
- Supports semantic actions (code snippets executed during parsing).

Challenges

- Learning curve for grammar syntax.
- Debugging grammar conflicts (e.g., shift/reduce errors).
- Generated code may be hard to customize deeply.

8. Automatic Generation of Parsers

Automatic generation of parsers is a cornerstone of modern compiler design and language processing. It allows developers to define a grammar and let a tool generate the parsing code—saving time, reducing errors, and ensuring consistency.

What Is Automatic Parser Generation?

Instead of manually writing a parser, you:

1. **Define a grammar** (usually in BNF or EBNF format).
2. **Feed it into a parser generator tool.**
3. The tool produces **source code** for a parser in your chosen language.

This parser can then analyze input strings, build parse trees, and even trigger semantic actions.

Popular Parser Generators

Tool	Language Support	Parser Type	Highlights
ANTLR	Java, Python, C#, etc.	LL(*)	Rich features, tree walking, listener/visitor patterns.
YACC/Bison	C/C++	LALR(1)	Classic tools, often paired with Lex for tokenizing.
PLY	Python	LALR(1)	Python-based Lex/YACC implementation.
PEG.js	JavaScript	PEG	Great for browser-based parsers.
Nearley	JavaScript	Earley	Handles ambiguous grammars well.

Work

1. **Grammar Definition:** You write rules like:
 $\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$
 $\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$
 $\text{factor} \rightarrow (\text{expr}) \mid \text{number}$
2. **Tool Processing:** The parser generator analyzes the grammar, builds parsing tables or recursive functions, and outputs code.
3. **Integration:** You use the generated parser in your application to process input and build syntax trees or execute actions.

Benefits

- **Speed:** Rapid parser development.
- **Accuracy:** Fewer bugs than hand-written parsers.
- **Maintainability:** Easy to update grammar rules.
- **Power:** Handles complex grammars and ambiguity.

Challenges

- **Learning Curve:** Grammar syntax and parser theory can be tricky.
- **Debugging:** Grammar conflicts (e.g., shift/reduce) may require deep understanding.
- **Customization:** Generated code may be hard to tweak manually.

9. Syntax-Directed Definitions

Syntax-Directed Definitions (SDDs) are a powerful framework in compiler design that link **grammar rules** with **semantic actions**. They allow you to define how meaning (semantics) is attached to the structure (syntax) of a language.

An **SDD** augments a context-free grammar by associating **attributes** and **semantic rules** with grammar productions. These attributes carry information like types, values, or code snippets, and the semantic rules define how to compute them.

Syntax-Directed Definitions bridge the gap between **syntax** and **semantics** by:

- Annotating grammar rules with **attributes**
- Using **semantic rules** to compute those attributes
- Enabling **meaningful translation** of source code during parsing

Components of an SDD

1. **Attributes:**
 - **Synthesized:** Computed from children nodes and passed up the parse tree.
 - **Inherited:** Passed down from parent or siblings in the parse tree.
2. **Semantic Rules:**
 - Define how attributes are computed.
 - Attached to grammar productions.

Example

Grammar:

Code

$E \rightarrow E1 + T$

$E \rightarrow T$

$T \rightarrow \text{num}$

SDD:

Code

$E.val = E1.val + T.val$

$T.val = num.lexval$

Here:

- val is a synthesized attribute.
- $num.lexval$ is the lexical value of a number token.

Types of SDDs

Type	Description
S-Attributed	Only synthesized attributes. Easy to implement with bottom-up parsers.
L-Attributed	Allows both synthesized and inherited attributes. Suitable for top-down parsers.

Applications

- **Type checking**
- **Intermediate code generation**
- **Symbol table management**
- **Constant folding and optimization**

10. Construction of Syntax Trees

A **syntax tree** (also called a **parse tree** or **abstract syntax tree (AST)**) is a graphical representation of the syntactic structure of a string according to a grammar.

- **Concrete Syntax Tree:** Shows every detail, including terminals and non-terminals.
- **Abstract Syntax Tree (AST):** Simplified version that omits unnecessary grammar details and focuses on the essential structure.

Components of a Syntax Tree

- **Nodes:** Represent grammar symbols (non-terminals and terminals).
- **Edges:** Show how symbols are derived from production rules.
- **Root:** The start symbol of the grammar.
- **Leaves:** Terminal symbols (tokens from the input).

Construction Process

1. Start with the Grammar

Example:

Code

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

2. Tokenize the Input

Input: $id + id * id$

Tokens: $id, +, id, *, id$

3. Apply Parsing Algorithm

Use a parser (e.g., recursive descent, LL, LR) to:

- Match tokens to grammar rules.
- Build tree nodes for each rule application.

4. Build Tree

Each production rule creates a node:

- $E \rightarrow E + T$ becomes a node with children E, +, T
- $T \rightarrow T * F$ becomes a node with children T, *, F

Example: AST for `id + id * id`

Code

```
+
/\
id *
/\
id id
```

This AST reflects operator precedence: * binds tighter than +.

Benefits of Syntax Trees

- **Semantic Analysis:** Type checking, scope resolution.
- **Code Generation:** Translate tree into machine or intermediate code.
- **Optimization:** Simplify expressions, eliminate redundancies.
- **Error Reporting:** Pinpoint syntax errors during parsing.

Tools That Build Syntax Trees

- **Parser Generators:** ANTLR, Bison, PLY
- **Compilers:** GCC, Clang
- **Interpreters:** Python, JavaScript engines

11. Bottom-Up Evaluation of S-Attributed Definitions

Bottom-up evaluation of **S-attributed definitions** is a technique used in **syntax-directed translation** during **bottom-up parsing**, such as with LR parsers. It focuses on computing **synthesized attributes**—values that are passed **upward** in the parse tree from children to parent nodes.

An **S-attributed definition** is a type of syntax-directed definition where:

- **All attributes are synthesized.**
- Each non-terminal computes its attribute **based on its children**.
- Ideal for **bottom-up parsers** because they build parse trees from leaves to root.

Bottom-Up Evaluation Strategy

During bottom-up parsing:

1. **Shift** tokens onto the stack.
2. When a **reduction** is applied (i.e., a production rule matches), compute the synthesized attribute for the left-hand side non-terminal.
3. Push the result back onto the stack.
4. Repeat until the entire input is reduced to the start symbol.

Example

Grammar:

Code

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow \text{num}$

Semantic Rules:

Code

$E.val = E1.val + T.val$

$T.val = num.lexval$

Input: 3 + 4

Evaluation Steps:

1. Shift 3 $\rightarrow num.lexval = 3 \rightarrow T.val = 3$
2. Reduce $T \rightarrow num \rightarrow E.val = 3$
3. Shift +
4. Shift 4 $\rightarrow num.lexval = 4 \rightarrow T.val = 4$
5. Reduce $T \rightarrow num$
6. Reduce $E \rightarrow E + T \rightarrow E.val = 3 + 4 = 7$

Advantages

- Perfect fit for **LR parsers** and other bottom-up strategies.
- Simple and efficient for arithmetic expressions and code generation.
- No need for inherited attributes or complex tree traversal.

Since bottom-up parsers reduce from leaves to root, synthesized attributes naturally flow upward—making S-attributed definitions a clean and compatible choice.

12. L-Attributed Definitions

L-attributed definitions are a type of syntax-directed definition used in compiler design to associate semantic meaning with grammar rules. They extend the simpler **S-attributed definitions** by allowing both **synthesized** and **inherited attributes**, making them more flexible and suitable for **top-down parsing**.

What Are L-Attributed Definitions?

An **L-attributed definition** satisfies the following condition:

- Each **inherited attribute** of a symbol in a production rule depends only on:
 - The attributes of symbols **to its left** in the production.
 - The attributes of the **parent non-terminal**.

This restriction ensures that attributes can be evaluated **left-to-right**, which aligns perfectly with **top-down parsers** like recursive descent.

Example

Grammar:

Code

$S \rightarrow A B$

$A \rightarrow a$

$B \rightarrow b$

Attributes:

- $A.in$ is inherited from $S.in$
- $B.in$ is inherited from $A.out$
- $S.out$ is synthesized from $B.out$

Semantic Rules:

Code

$A.in = S.in$

$B.in = A.out$

$S.out = B.out$

This setup ensures that each attribute is computed using only information available from the left or the parent—making it L-attributed.

Evaluation Strategy

- **Top-down parsers** evaluate attributes as they expand productions.
- Inherited attributes are passed **down** from parent or left siblings.
- Synthesized attributes are passed **up** from children.

Advantages

- More expressive than S-attributed definitions.
- Compatible with **top-down parsing**.
- Supports semantic tasks like type checking, symbol table management, and parameter passing.

Limitations

- Still not general enough for all grammars.
- Cannot handle arbitrary inherited dependencies (e.g., right-to-left).

Summary

Feature	L-Attributed Definitions
Attribute Types	Synthesized + Inherited
Evaluation Direction	Left-to-right
Parser Compatibility	Top-down (e.g., recursive descent)
Use Cases	Type checking, parameter passing

13. syntax directed definitions and translation schemes

An **SDD** augments a context-free grammar by attaching **attributes** and **semantic rules** to grammar productions.

Components:

- **Attributes:** Values associated with grammar symbols.
 - **Synthesized:** Computed from children nodes (bottom-up).
 - **Inherited:** Passed from parent or siblings (top-down).
- **Semantic Rules:** Define how attributes are computed.

Example:

Grammar:

Code

$E \rightarrow E + T$

$T \rightarrow \text{num}$

SDD:

Code

$E.\text{val} = E1.\text{val} + T.\text{val}$

$T.\text{val} = \text{num.lexval}$

This computes the value of an arithmetic expression during parsing. Types of SDDs:

Type	Description	Parser Compatibility
S-Attributed	Only synthesized attributes	Bottom-up (LR)
L-Attributed	Synthesized + inherited (left-to-right)	Top-down (LL)

Syntax-Directed Translation Schemes (SDTs)

An **SDT** is a practical implementation of an SDD where **semantic actions** are embedded directly into grammar productions.

Format:

Each production includes **code snippets** (actions) that execute during parsing.

Example:

Code

$E \rightarrow E + T \{ E.\text{val} = E.\text{val} + T.\text{val}; \}$

$T \rightarrow \text{num} \{ T.\text{val} = \text{num.lexval}; \}$

These actions are executed when the parser applies the corresponding production.

Types of SDTs:

Type	Description
Postfix SDT	Actions placed at the end of productions
Embedded SDT	Actions placed anywhere in the production

Applications

- **Expression evaluation**
- **Type checking**
- **Intermediate code generation**
- **Symbol table management**

Summary

Feature	SDD	SDT
Purpose	Define semantics via attributes	Implement semantics via actions
Structure	Attribute rules per production	Embedded code in grammar
Evaluation	Attribute computation	Action execution during parsing
Parser Compatibility	LL (L-attributed), LR (S-attributed)	Depends on action placement
